DHAR

Bare Metal. Absolute Control.

VOLUME II — ENGINEERING ROADMAP

STAGE-BETA

"Honing Dhar — the compiler proves itself by rebuilding itself."

DOCUMENT	Stage-beta — Honing Dhar
VERSION	1.1
DATE	October 26, 2026
STATUS	Ratified
PART OF	The Canon of Dhar, Volume II
DEPENDS ON	Spec0, Spec1, Stage-alpha, Amendments A-1 and A-2
PRECEDES	Stage-gamma — Optimizing Dhar

1. Mandate

Stage-beta is the first real Dhar compiler: a complete lexer, parser, type checker, and code generator, written entirely in Dhar, using only the bootstrap subset Stage-alpha can compile. Its defining test is not that it works, but that it can compile itself.

Dhar is a language for trustworthy computation: it goes first to cybersecurity, then to artificial intelligence, then to quantum computing, with every other domain kept as modules (Spec0 §5.1, §5.2; Amendment A-1). Stage-beta is where that direction is first made structural. The features of the three tracks arrive only after Stage-beta compiles itself (Amendment A-1 §7), but what they need from the compiler is designed into Stage-beta's intermediate representation now, because it is cheap here and expensive after an optimizer exists (§3.1). Stage-beta begins when Stage-alpha is complete on every committed target (Amendment A-2 §2).

2. Scope

2.1 In scope

- The full language and grammar of Spec1 v3.1: the 46 keywords (Spec1, in full). The additions of Spec1 v3.2 (numeric types for models and cryptography, secret values, arenas: Spec1 §4.7, §14.5, §14.6) are delivered by the tracks after Stage-beta compiles itself; Stage-beta leaves room for them (§3.1).
- Static type inference (Spec1 §4.3) and the wild dynamic mode (§4.4).
- The complete control-flow set: branching, loops, dispatch (Spec1 §8).
- mold/forge, instantiation, and the OOP/functional/logical/symbolic mechanisms (Spec1 §9–10).
- Collections: literal syntax and type names (Spec1 §13).
- The memory model: ownership, view/grab, raw memory (Spec1 §14).
- Error handling: state/trap/enforce and attempt/rescue/ensure/raise (Spec1 §16).
- Modules and visibility (Spec1 §17).
- Universal IR generation (Spec0 §4) and **a back end for every committed target** (Amendment A-2 §3.1), each re-implemented on the IR and each held to the same conformance battery as Stage-alpha's back end for that target (§4.2).

2.2 Out of scope

- Optimization passes of any kind — Stage-beta's code generation is expected to be direct and unoptimized. That is Stage-gamma's mandate, not this one.
- The standard library beyond what Stage-beta needs to build itself.
- Targets outside the committed list of Amendment A-2 §3.1; a new target is added by revising that amendment.
- The features of the security, AI and quantum tracks themselves (Amendment A-1 §4 to §6), apart from the room left for them in §3.1.

3. Required Architecture

MODULE	RESPONSIBILITY
lexer.dhar	Tokenizing, keyword recognition (Spec1 §2)
parser.dhar	Recursive-descent parsing into an AST (Spec1, full grammar)
ast.dhar	AST node types — the first real use of mold for the compiler's own data (Spec1 §9)
symbol.dhar	Symbol table and scope resolution
typecheck.dhar	Type inference and checking (Spec1 §4.3–4.4)
ir.dhar	Universal IR generation (Spec0 §4)
codegen.dhar	Back-end emission from the Universal IR
target/*.dhar	One target description table and one small emitter per committed target (§3.1, R4)
main.dhar	The driver — see the rule below

`main.dhar` **must** `pull` and integrate every other module into one working pipeline. A driver file that declares a pipeline but does not call into the modules that implement it is not a driver — this requirement exists specifically so that Stage-beta cannot be declared complete while its modules remain wired together only in comments.

3.1 The intermediate representation: what it must leave room for

These are design requirements, not features (Amendment A-1 §8). Each is cheap in the first intermediate representation and expensive to add after an optimizer exists. Stage-beta's front end does not yet produce the types and attributes below; the IR **must** be able to carry them.

1. **R1. Typed values wide enough for the tracks.** The IR's type set includes integer widths from 8 to 128 bits, 16-bit floating point in both formats (`f16`, `bf16`), `f32`, `f64`, fixed-width vectors and a complex pair (Spec1 §4.7).
2. **R2. A secret attribute the optimizer must respect.** Values and memory regions can be marked secret, and the IR records it (Spec1 §14.5). The passes of Stage-gamma may not turn a select into a branch on a secret, may not introduce a table lookup indexed by one, and may not delete the clearing of a secret buffer (Stage-gamma §3.1).
3. **R3. No hidden allocation.** Every allocation is visible in the IR and belongs to a region, so memory use can be bounded and arenas are natural (Spec1 §14.6).
4. **R4. Targets described by tables.** The back ends read a description of each processor: vector width, cryptographic instructions (AES, SHA, carry-less multiply, and the cryptography extensions of ARM and RISC-V), alignment rules. A new processor is a new table and a small emitter, not a new compiler. Stage-alpha reaches the same targets without an intermediate form (Amendment A-2); Stage-beta's back ends are re-implemented on the IR and must pass the same tests (§4.2).
5. **R5. Stage-beta stays inside what Stage-alpha accepts** until the handoff, as the bootstrap requires. The tracks add nothing to Stage-beta's scope except R1 to R4.

4. Acceptance Criteria

This stage has exactly one criterion that matters, stated precisely because it is easy to convince oneself a compiler is self-hosting when it is not:

```
Let S = Stage-beta's complete source.
1. Stage-alpha compiles S ⟶ E1 (generation 1)
2. E1 compiles S again ⟶ E2 (generation 2)
3. compare(E1, E2) byte-for-byte, ignoring timestamps
E1 ═ E2 ⇒ Stage-beta is self-hosting.
E1 ≠ E2 ⇒ Stage-beta is not complete, regardless of
                  how much of the language appears to work.
```

Two supporting rules close the loopholes a partial implementation could otherwise hide behind:

1. Every construct in Spec1 (v3.1; §2.1) must have at least one passing test compiled through this exact pipeline. A construct with no test is not considered implemented, no matter how much code exists for it.
2. A pipeline stage that prints progress messages without acting on real input — a lexer that tokenizes without a following parser consuming the tokens, or a driver that prints “Symbols: 0” regardless of the source — does not satisfy this stage's mandate even if it runs without crashing.

4.1 The build gates (Amendment A-1, S4)

Self-hosting says the compiler can reproduce itself. It does not say the compiler can be trusted, because a corrupted compiler reproduces a corrupted copy of itself perfectly (Tempering Dhar §5.3). Four conditions, each tested, close that gap; together they are gate S4 (Amendment A-1 §9):

1. **Fixed point.** The criterion above: E1 and E2 are byte-for-byte identical.
2. **Two independent compilers.** Stage-alpha is written in assembly and Stage-beta in Dhar, so they are independent implementations. Stage-alpha compiles Stage-beta's source (E1), E1 compiles it again (E2), and **E2 must equal the Stage-beta binary that is distributed**. Equality shows that the distributed binary came from the published source and carries no hidden change (the check known as diverse double-compilation), unless Stage-alpha itself is compromised. The check requires Stage-alpha to be kept frozen and archived, not improved (§5).
3. **Reproducible.** Two machines, the same source and the same flags produce identical output.
4. **Listed.** Every build emits a software bill of materials, and releases are signed (Stage-epsilon §3).

The test for S4: build the compiler twice on one machine and once on a second machine, and compare the three outputs byte for byte; all three must be identical and the bill of materials complete.

4.2 The back ends on the intermediate representation

For every committed target (Amendment A-2 §3.1), the Stage-beta back end **must** pass the same conformance battery, with the same expected results, as Stage-alpha's back end for that target (Stage-alpha §3.3), on the same kind of real system (gate G1). The output of Stage-alpha's back ends is the reference against which each Stage-beta back end is compared (Amendment A-2 §7). Stage-beta is complete when its back ends for every committed target pass, in addition to the criterion above.

5. Handoff to Stage-gamma

Stage-alpha is frozen and archived at the handoff from Stage-alpha (Stage-alpha §5) and stays so: it is the second compiler of §4.1, and it **must not** be improved. Once self-hosting is proven and the gate of §4.1 and the back ends of §4.2 pass, Stage-gamma **may** begin, but it **must not** begin until Spec1's syntax has been stable — no ratified changes — for a full development cycle, so optimization work is not built against a moving target. Stage-gamma works on the IR of §3.1 and keeps its invariants (Stage-gamma §3.1).

Revision log

Stage-beta — Honing Dhar · v1.1 · October 26, 2026 · Ratified.

VERSION	DATE	CHANGE
v1.0	September 6, 2026	First ratified edition.
v1.1	October 26, 2026	Amendment A-1: design requirements R1 to R5 for the intermediate representation (§3.1) and the build gates S4 (§4.1). Amendment A-2: back ends for every committed target re-implemented on the IR and held to the same battery (§2.1, §4.2); a frozen Stage-alpha (§5). Cross-references to Spec1 corrected to the current section numbers.